ESCRIU LA TEVA PRIMERA APP AMB FLUTTER



Eduard Carreras

Desenvolupament d'Aplicacions mòbils per iOS i Android amb Flutter
Activitat d'Aprenentatge 1.2

ÍNDEX

INTRODUCCIÓ 2
QUÈ APRENDRÀS? 2
QUÈ CONSTRUIRÀS? 2
ENTORN 2
CREACIÓ DE L'APLICACIÓ FLUTTER INICIAL 3
UTILITZACIÓ DE PAQUETS EXTERNS 5
AFEGIR UN WIDGET AMB ESTAT 7
CREAR UN LISTVIEW DE DESPLAÇAMENT INFINIT 11
TASQUES ADDICIONALS (OPCIONAL) 15
REFERÈNCIES 15

INTRODUCCIÓ

En aquesta activitat d'aprenentatge, crearàs una aplicació mòbil senzilla en Flutter. Si estàs familiaritzats amb el codi orientat a objectes i conceptes bàsics de programació, com ara variables, bucles i condicionals, podràs completar l'Activitat d'Aprenentatge.

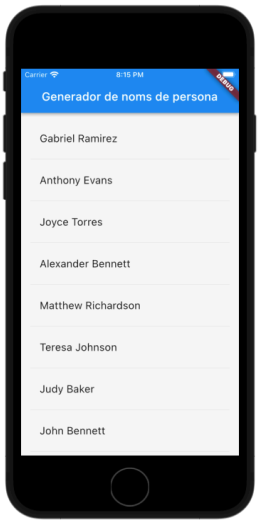
Farem servir coses que encara no hem vist a classe; no cal que ho entenguis tot ara, ho anirem veient a les properes sessions. No cal experiència prèvia amb programació de Dart, mòbil, d'escriptori o web. L'activitat et guiarà pas a pas.

QUÈ APRENDRÀS?

- Com escriure una aplicació Flutter que funcioni a iOS i Android
- Estructura bàsica d'una aplicació Flutter
- Trobar i utilitzar paquets per ampliar la funcionalitat
- A utilitzar el hot reload per a un cicle de desenvolupament més ràpid
- Com implementar un widget amb estat
- Com crear una llista infinita que s'omple de manera dinàmica

QUÈ CONSTRUIRÀS?

Implementarem una aplicació senzilla que generi noms de persones. El codi genera els noms que apareixen a la pantalla. A mesura que l'usuari es desplaça, es generen més noms. No hi ha límit a la distància que un usuari pot desplaçar.



Aquesta imatge mostra com l'app quedaria en finalitzar l'activitat.

ENTORN

Per completar l'activitat necessiteu:

1. Tenir instal·lat el Flutter SDK
2. Tenir instal·lat Visual Studio Code
3. Una d'aquestes tres opcions:
    a. Un dispositiu físic Android o iOS connectat a l'ordinador i configurat en

mode de desenvolupador
b. El simulador d'iOS (requereix la instal·lació d'eines Xcode)
c. L'emulador d'Android (requereix configuració a Android Studio)

CREACIÓ DE L'APLICACIÓ FLUTTER INICIAL

Crea un projecte Flutter:

- A Visual Studio Code, obre la paleta d'ordres (amb `F1`, `Ctrl+Maj+P` o `Maj+Cmd+P`) i escriu i selecciona "**Flutter: New Project**" i selecciona el tipus **"Application"**

El primer que farem és editar l'exemple que es crea per defecte.

Edita el contingut de `lib/main.dart`. Suprimeix **tot el codi** de `lib/main.dart` i **substitueix-lo** pel codi següent, que mostra "Hola món" al centre de la pantalla:

```dart
import 'package:flutter/material.dart';

void main() {
 runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});

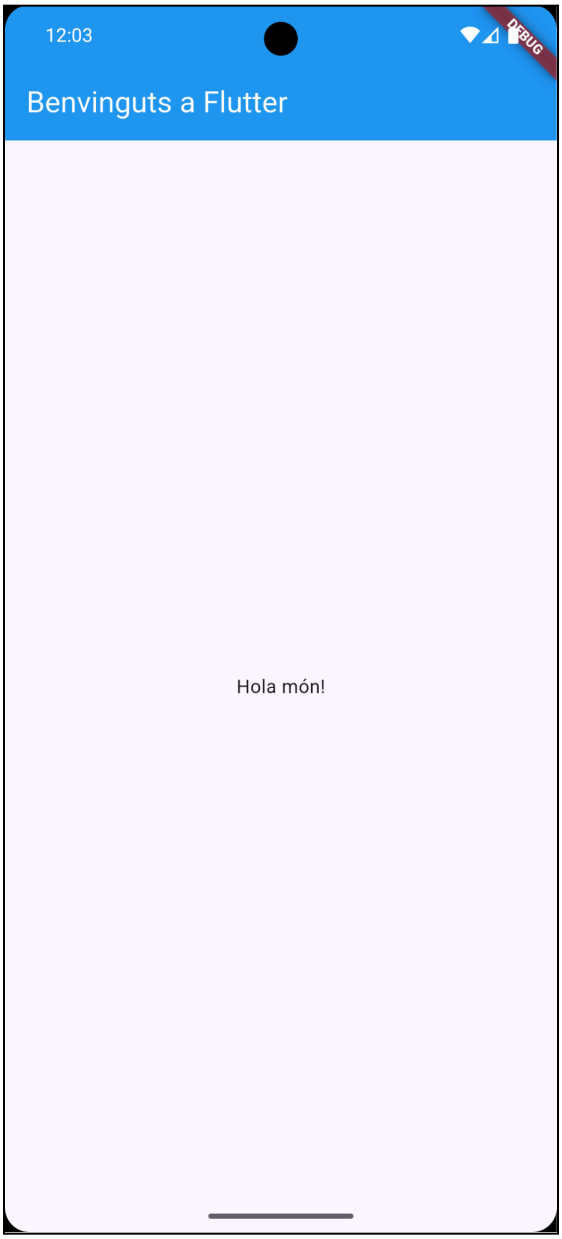
 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     title: 'Flutter Demo',
     home: Scaffold(
       appBar: AppBar(
         backgroundColor: Colors.blue,
         foregroundColor: Colors.white,
         title: const Text('Benvinguts a Flutter'),
       ),
       body: const Center(child: Text('Hola món!')),
     ),
   );
 }
}
```



En enganxar el codi, la indentació podria no quedar alineada.

> Per formatar el codi a Visual Studio Code, feu servir Maj+Alt+F (Windows i Linux) o Maj+Option+F (Mac).

Arrenca un emulador o connecta un dispositiu físic. Selecciona aquest dispositiu i executa l'app. S'hauria de veure una cosa així:





La primera vegada que utilitzeu un dispositiu físic, pot trigar una estona a carregar-se. Després, pots utilitzar *Hot Reload* per a actualitzacions ràpides. Guardant el nostre codi també es realitza un *hot reload* si l'aplicació s'està executant.

Observacions

Aquest exemple crea una aplicació que utilitza la llibreria `material`, que conté widgets que utilitzen Material Design. Material Design és un llenguatge de disseny visual de Google que és estàndard a mòbil i web. Flutter ofereix un ampli conjunt de widgets de Material Design.

La classe de l'aplicació (`MyApp`) estén de la classe `StatelessWidget` cosa que fa que l'aplicació sigui un widget. A Flutter, gairebé tot és un widget, fins i tot `Center`, `Padding` o `Layout`.

El widget `Scaffold`, de la llibreria `material`, proporciona una barra d'aplicacions predeterminada, un títol i una propietat `body` que conté l'arbre de widgets per a la pantalla d'inici. A l'exemple només tenim un text, però aquest arbre pot arribar a ser molt complex.

El treball principal d'un widget és proporcionar un mètode `build` que defineix com es mostra el widget en termes d'altres widgets de nivell inferior.

El `body` d'aquest exemple consisteix en un widget `Center` que conté un widget `Text` com `child.` El widget `Center` alinea el seu `child` al centre de la pantalla.

UTILITZACIÓ DE PAQUETS EXTERNS

En aquest pas, utilitzaràs un paquet de codi obert anomenat `random_name_generator`, que serveix per generar a l'atzar noms de persones.

Pots trobar el paquet `random_name_generator`, així com molts altres paquets de codi obert, a https://pub.dev/.

Afegirem el paquet `random_name_generator` com a dependència d'aquesta aplicació. Per fer-ho, obrirem la pestanya `TERMINAL` a la part inferior del nostre entorn de desenvolupament i escrivim aquesta comanda:

```
flutter pub add random_name_generator
```



Amb la comanda ***flutter pub add*** podem afegir qualsevol paquet que estigui en el repositori ***pub.dev***. Només ens cal saber el nom del paquet.

Obrim l'arxiu `pubspec.yaml`, que és l'arxiu de configuració de l'aplicació, i comprovem com aquesta instrucció ha afegit aquesta línia a l'apartat `dependencies:`

```
random_name_generator: ^1.5.0
```

Ara, a `lib/main.dart` afegim una línia als imports:

```
import 'package:flutter/material.dart';
import 'package:random_name_generator/random_name_generator.dart';
```

Observa com a mesura que escrius, se t'ofereixen suggeriments per importar llibreries. A continuació, mostrarà la línia de codi d'importació amb un warning i, passant el cursor per sobre, us informarà que la llibreria importada no s'utilitza (de moment).

Ara utilitzarem el paquet `random_name_generator` per generar un nom aleatori on ara diu "Hola món".

Farem els següents canvis a la nostra app:

```
@override
Widget build(BuildContext context) {
  final randomNames = RandomNames(Zone.us); // Afegeix aquesta línia
  return MaterialApp(
    title: 'Flutter Demo',
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Benvinguts a Flutter'),
      ),
      body: Center(child: Text(randomNames.fullName())), // Modifica el body
    ),
  );
```

```
}
```



Si veieu que l'avaluació del widget Text dona un error, probablement us heu oblidat d'eliminar `const` del widget `Center`

Si l'aplicació s'està executant, podeu forçar un *hot reload*  per actualitzar-la. Cada vegada que feu clic a *hot reload* o guardis el projecte, hauries de veure un nom diferent, escollit a l'atzar, a l'aplicació en execució. Això és perque la generació de noms es genera dins del mètode `build`, que s'executa cada vegada que es construeix el widget.

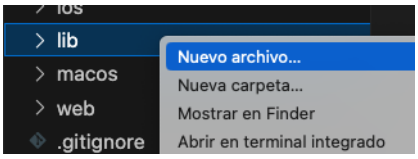
AFEGIR UN WIDGET AMB ESTAT

Els `StatelessWidget` són immutables, cosa que significa que les seves propietats no poden canviar; tots els valors són finals.

En canvi, els `StatefulWidget` mantenen un estat que pot canviar durant la vida útil del widget. La implementació d'un widget amb estat requereix almenys dues classes, una classe `StatefulWidget` que conté una instància d'una classe `State`. L'objecte `StatefulWidget` és, per si mateix, immutable, però l'objecte `State` evoluciona durant la vida útil del widget.

En aquest pas, afegiràs un `StatefulWidget`, que anomenarem `PeopleList`, que crea la seva classe `State`, que es dirà `_PeopleListState`. A continuació, utilitzaràs `PeopleList` com a `child` dins del widget `MyApp`.

Per mantenir un codi net és convenient tenir cada classe en un fitxer separat. Crearem un nou arxiu `people_list.dart` al directori `lib` on guardarem el nostre widget. Per això cliquem al directori `lib` amb el botó dret i seleccionem *Nou Arxiu*:





Observem les *naming conventions* de Dart: mentre que els noms de

> les classes comencen sempre amb majúscules (`PeopleList`), els noms de fitxer van sempre en minúscula i utilitzant el _ com a separador de paraules (`people_list.dart`)

Situem-nos ara al nou arxiu creat, i comencem a escriure les lletres `stful`. A mesura que anem escrivint, sense que calgui acabar d'escriure-ho, se'ns suggerirà crear un nou `StatefulWidget`. Acceptant la suggerència, es crearà tota l'estructura d'un `StatefulWidget`. Just quan aparegui, podem Introduir el nom del widget: `PeopleList`.

Quan hagueu introduït `PeopleList` com a nom del `StatefulWidget`, el nostre IDE actualitza automàticament la classe `State` que l'acompanya, anomenant-la `_PeopleListState`. Per defecte, el nom de la classe `State` té el prefix d'un guió baix. Prefixar un identificador amb un guió baix fa que l'objecte sigui privat (és a dir que no es vegi fora d'aquest fitxer) i és una pràctica recomanada per als objectes `State`.

També s'actualitza automàticament la classe d'estat com `State<PeopleList>`, indicant que utilitzeu una classe `State` genèrica especialitzada per utilitzar-la amb un objecte del tipus `PeopleList`. La major part de la lògica de l'aplicació resideix aquí mantenint l'estat del widget `PeopleList`. Aquesta classe mostrarà una llista de noms generats, que creixerà infinitament a mesura que l'usuari es desplaça.

Ara les dues classes tenen el següent aspecte:

```
import 'package:flutter/material.dart';

class PeopleList extends StatefulWidget {
  const PeopleList({super.key});

  @override
  State<PeopleList> createState() => _PeopleListState();
}

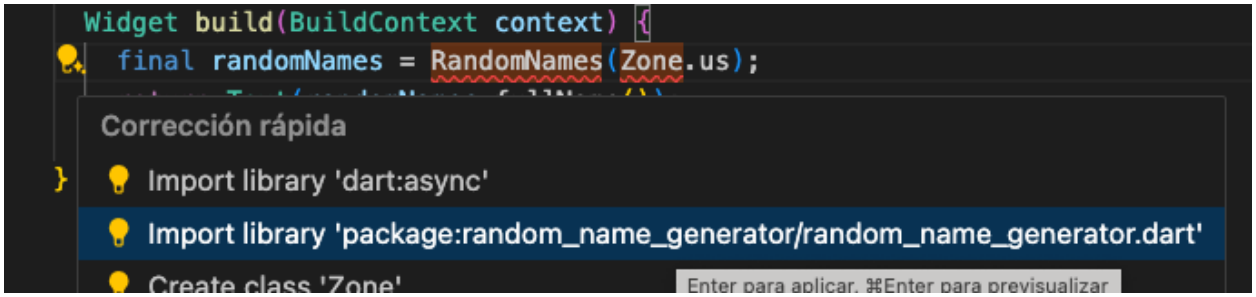
class _PeopleListState extends State<PeopleList> {
  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

En aquest punt modificarem el contingut del mètode `build` de la classe

`_PeopleListState`, afegint tot el que volem que aparegui:

```
class _PeopleListState extends State<PeopleList> {
  @override
  Widget build(BuildContext context) {
    final randomNames = RandomNames(Zone.us);
    return Text(randomNames.fullName());
  }
}
```

Fixa't que, en escriure aquest codi, tindràs un error. Això és perquè no hem importat la classe `RandomNames`. Això és molt senzill de solucionar, clicant en la bombeta que apareixerà a la dreta del codi si ens posicionem sobre l'avís. Ens donarà alternatives de la llibreria que hem d'importar i ho afegirà al codi de forma automàtica:



Ara, a `main.dart`, eliminarem el codi de generació de noms de la classe `MyApp` i apuntarem al nou widget que hem creat.

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final randomNames = RandomNames(Zone.us); // Eliminem aquesta línia
    return MaterialApp(
      title: 'Flutter Demo',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Benvinguts a Flutter'),
        ),
        body: PeopleList(), // Modifica el body
      ),
```

```
    );
  }
}
```

Revisa els *import*, que s'hauran de modificar (afegir-ne un i eliminar-ne un altre, però és possible que l'IDE l'afegeixi automàticament).

Torna a fer un *hot reload*. L'aplicació ha de comportar-se com abans, mostrant un nom aleatori de persona cada cop que recarregueu o deseu l'aplicació.

CREAR UN LISTVIEW DE DESPLAÇAMENT INFINIT

En aquest pas, s'ampliarà `_PeopleListState` per generar i mostrar una llista de noms de persones. A mesura que l'usuari es desplaça, la llista (que es mostra en un widget `ListView`) creix infinitament. El constructor `builder` de `ListView` us permetrà crear una vista de llista a demanda.

Afegirem algunes variables d'estat a la classe`_PeopleListState`. Aquestes variables les hi posarem just al principi del contingut de la classe `_PeopleListState`.

Les tres variables que afegirem son:

- Una variable `_suggestions` que contindrà una llista de les combinacions de noms generats.
- Una variable `_biggerFont` que contindrà l'estil de text per augmentar la mida del tipus de lletra.
- Una variable per al generador de noms `_randomNames`, que serà l'objecte encarregat de generar els noms aleatoris.

```
class _PeopleListState extends State<PeopleList> {
 final _suggestions = <String>[];                    // Nou
 final _biggerFont = const TextStyle(fontSize: 18); // Nou
 final _randomNames = RandomNames(Zone.us);          // Nou
 ...
}
```

A continuació, actualitzarem el mètode `build` de la classe `_PeopleListState`.

El widget `ListView` s'utilitza a Flutter per mostrar una llista de widgets un a sobre de l'altre, de manera que es pugui fer *scroll*. Aquest widget té un constructor `ListView.builder` que ens facilita molt les coses a l'hora d'implementar els elements de la llista.

Al constructor `ListView.builder` li passem el paràmetre `itemBuilder`, que és una funció *callback* que definim a partir de dos paràmetres: el `BuildContext` actual (que es refereix a la pantalla on estem) i un valor `i` que és l'índex de l'element que estem pintant. Flutter crida aquesta funció per a cada fila que ha de mostrar, i li passa el número de fila `i`. Aquest model permet que la llista de suggeriments segueixi creixent a mesura que l'usuari es desplaça.

```
@override
Widget build(BuildContext context) {
  return ListView.builder(
   itemBuilder: (context, i) {
     if (i >= _suggestions.length) {
       //Afegim 10 noms més
       for (var j = 0; j < 10; j++) {
         _suggestions.add(_randomNames.fullName());
       }
     }
     return ListTile(
       title: Text(
         _suggestions[i],
         style: _biggerFont,
       ),
     );
    },
  );
}
```

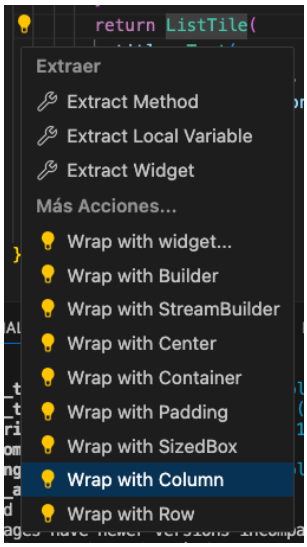
La funció `itemBuilder` es cridarà una vegada per cada element del `ListView` i col·locarà per cada nom un widget del tipus `ListTile` que conté el nom generat.



El codi a l'inici de la funció `itemBuilder` serveix per generar dinàmicament més noms aleatoris si l'índex de l'element que es

> mostra a la pantalla és més gran que la llista d'elements generats, és a dir, si encara no s'ha generat aquest element.

Ara afegirem un separador entre els elements. Per fer això, *embolicarem* el widget `ListTile` en una columna (un widget `Column`). Aquest pas (embolicar en un altre widget) el podem fer de forma ràpida fent servir altre cop la bombeta. Ens situarem al widget que volem embolicar (`ListTile`) i clicarem a la bombeta, seleccionant *Wrap with Column*:



El nostre `ListTile` quedarà embolicat pel widget `Column`:

```
return Column(
 children: [
   ListTile(
     title: Text(
       _suggestions[i],
       style: _biggerFont,
     ),
   ),
 ],
);
```

És molt important dominar aquesta manera de treballar ja que ens agilitzarà molt el procés d'afegir i eliminar widgets dins de l'arbre.

Ara, la columna que hem creat només té un element per a cada nom (el `ListTile`). Posarem una coma al final del `ListTile` i hi afegirem a sota un widget `Divider`:

```
return Column(
  children: [
    ListTile(
      title: Text(
        data: _suggestions[i],
        style: _biggerFont,
      ),  // Text
    ),  // ListTile
    const Divider(),
  ],
);  // Column
```

Ja només ens queda actualitzar `MyApp` per canviar el títol en dos llocs:

```
class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
    title: 'Generador de noms',
     home: Scaffold(
       appBar: AppBar(
         backgroundColor: Colors.blue,
         foregroundColor: Colors.white,
        title: const Text('Generador de noms'),
       ),
       body: const Center(child: PeopleList()),
     ),
   );
 }
}
```

Desa el fitxer. Els títols canvien, però fixa't en els noms: **són els mateixos d'abans**. Fes uns quants *hot reload* més. Els noms tampoc canvien. Al pas anterior, en canvi, cada *hot reload* en mostrava un de nou. Per què?

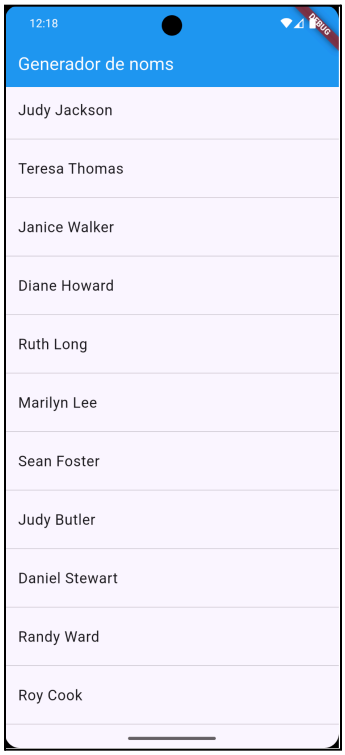


Al pas anterior, el nom es creava **dins del mètode `build`**, i el *hot reload* torna a executar `build`: per això cada cop sortia un nom nou. Ara els noms es guarden a la llista `_suggestions`, que viu dins de l'objecte `State`. El *hot reload* torna a executar `build`, però **conserva l'estat**: la llista ja existeix i es tornen a mostrar els mateixos noms. Aquesta és precisament la

feina d'un `StatefulWidget`: recordar dades entre una construcció i la següent.

Per començar de zero cal un ***hot restart*** (la fletxa circular verda de la barra de depuració, o Ctrl+Maj+F5 / Cmd+Maj+F5), que torna a crear l'app i el seu estat. Fes-ne un: ara sí que hauries de veure una llista nova de noms, i en desplaçar-te en sortiran més, sense límit.

Hauries de veure una llista de noms generats a l'atzar, independentment de la distància que es desplaci.



TASQUES ADDICIONALS (OPCIONAL)

A partir de l'app que has realitzat, et proposo fer algun dels següents reptes:

- Mira la documentació de la llibreria per generar noms catalans
- Alterna el color de fons de les files parells i senars
- Personalitza l'aspecte de l'app canviant colors o el disseny de pantalla
- En tocar un nom mostrar-lo en un `SnackBar`
- Substituir la `Column` + `Divider` per `ListView.separated`

REFERÈNCIES

1. Paquet `random_name_generator`: https://pub.dev/packages/random_name_generator